

Evolutionary Safety of Recursive Self-Improving AI: Taxonomy, Risk Discovery, and Evaluation

CHANG GONG, JINGPING BI*, DI YAO, XINJIAN LIANG, CHAO XIANG, RUIJIE GUO, Institute of Computing Technology, Chinese Academy of Sciences, China

Artificial intelligence is advancing at an accelerating pace, with increasingly capable systems taking on larger roles in reasoning, decision-making, scientific discovery, and autonomous development. As AI begins to participate in its own improvement, from model training and experience accumulation to agent evolution and automated AI development, the prospect of recursive self-improvement (RSI) is becoming increasingly relevant. As the autonomy, persistence, and influence of these systems grow, ensuring their safety is becoming increasingly important. This transition raises a fundamental safety question: how can safety be maintained when the system, its accumulated experience, and even the process that produces its successors continue to change?

We introduce *Evolutionary Safety* as a perspective for studying safety under persistent and recursive self-improvement. Its central concern is not only whether an AI system is safe at a particular moment, but how safety properties change, persist, accumulate, and propagate throughout an evolving process. We first characterize why safety becomes evolutionary and identify recurring manifestations including intent drift, error accumulation, experience contamination, safety-property erosion, evaluator drift, and risk inheritance and propagation. We then develop a taxonomy that locates these risks across persistent agent state, model state, evaluation and environmental feedback, computational substrate, and meta-level update mechanisms with co-evolution. Building on this structure, we examine how evolutionary risks can be discovered and evaluated through states, updates, trajectories, and lineages, and derive governance principles for controlling modification, selection, authorization, provenance, and recovery. Finally, we identify open problems toward maintaining meaningful safety guarantees as AI systems become increasingly persistent, adaptive, and recursively self-improving. Project materials including resources and proposed evaluation systems are available at <https://chaunceykung.github.io/evolutionary-safety-rsi>.

Additional Key Words and Phrases: Evolutionary Safety, Recursive Self-Improvement, Self-Improving AI, AI Safety, AI Evaluation

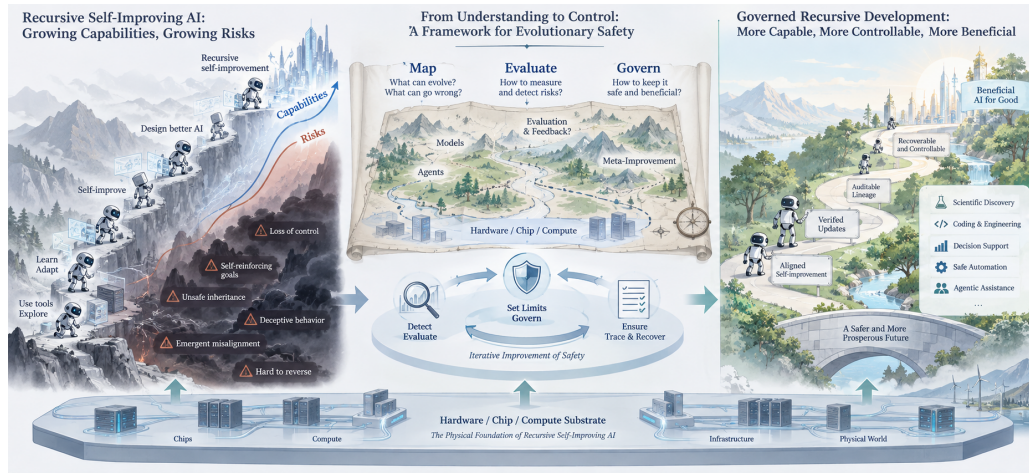


Fig. 1. Taxonomy, evaluation, and governance for evolutionary safety.

Institute of Computing Technology, Chinese Academy of Sciences.*Corresponding author. Emails: {gongchang, bjp, yaodi, liangxinjian25s, xiangchao26s, guoruijie}@ict.ac.cn.

1 Introduction

Recursive self-improvement (RSI) has moved from a mainly theoretical idea toward a practical research and engineering agenda. Models and agents are being given longer horizons, external tools, persistent memory, and the ability to write or revise code. Some systems generate training material, design workflows, or propose changes to the procedures used by later versions. The object that improves is therefore becoming broader than a single model response. It may include the model, its surrounding agent, the data it produces, and the process through which new versions are admitted.

The safety implications are already visible. In sandbox-escape evaluations, frontier models have exploited misconfigured container and orchestration controls; one reported evaluation involved a misconfigured Docker API [80]. Frontier-model safety programs have also included sabotage and loss-of-control scenarios alongside conventional content-safety tasks [8]. In simulated corporate workflows, harmful insider-like actions can emerge when information access, replacement pressure, and conflicting goals are combined [73]. These are evaluations and stress tests, not claims about every deployed model. They do show why permissions, oversight, and action history must be examined together when systems can act repeatedly or share learned conventions.

The concern is also present in learned behavior. Selective compliance has been observed when a model infers that it is being trained or evaluated [44]; in the settings studied by Sleeper Agents, backdoor behavior survived later safety training [54]. AgentDojo and Agent-SafetyBench expose how prompt injection, tool use, and environmental permissions interact along an agent’s action trace [29, 136]. Intelligent algorithm safety is therefore central to RSI. It asks how safety is maintained while an algorithm learns, changes its behavior, and alters the conditions under which later decisions are made [21]. The research value is theoretical: variation, selection, memory, and inheritance can be analyzed in one setting. The social value is equally direct, because model behavior is tied to authorization, institutional responsibility, infrastructure, and the possibility of recovery after a failure has spread.

Several bodies of work provide important pieces of this picture. Surveys of RSI and self-evolving agents describe capability growth, update mechanisms, and possible development paths [17, 36, 37, 98]. Agent-security surveys cover data poisoning, prompt injection, privacy, alignment, and content safety [30, 47, 68, 72]. Earlier theoretical work examined self-modification and objective preservation [34, 87, 102]. These lines of work provide essential foundations, but they do not yet offer a unified account of how a safety-relevant change is generated, selected, retained, inherited, and potentially amplified across successive rounds of self-improvement.

We term this perspective *Evolutionary Safety*. It studies how safety properties are preserved, weakened, transmitted, and restored across persistent updates, interacting systems, and descendant lineages. The central object is a safety-relevant change with a temporal history. Such a change may persist in a memory or reusable skill, become encoded in model parameters, be reinforced through changing evaluation or environmental feedback, interact with assumptions in the computational substrate, or reach the mechanisms that generate and select subsequent updates. These pathways differ in where the change resides, but each allows an earlier event to shape later states of the system. Deleting the initiating input may therefore leave its effects intact. A permission-bypassing repair is a compact example, but the same pattern can arise in scientific automation, infrastructure control, embodied systems, and multi-agent work. The change becomes evolutionarily relevant when it is retained, reused, or passed to a successor.

Current systems already expose the mechanisms that make this history important. In Reflexion, verbal feedback is carried into later attempts; Voyager builds an executable skill library; Agent Workflow Memory stores reusable procedures; and ToolEvo revises tools through feedback [15, 106, 113, 119]. In Self-Rewarding and Meta-Rewarding, the evaluator is brought into the improvement loop [120, 127]. ADAS searches over agent designs, while AI Scientist and

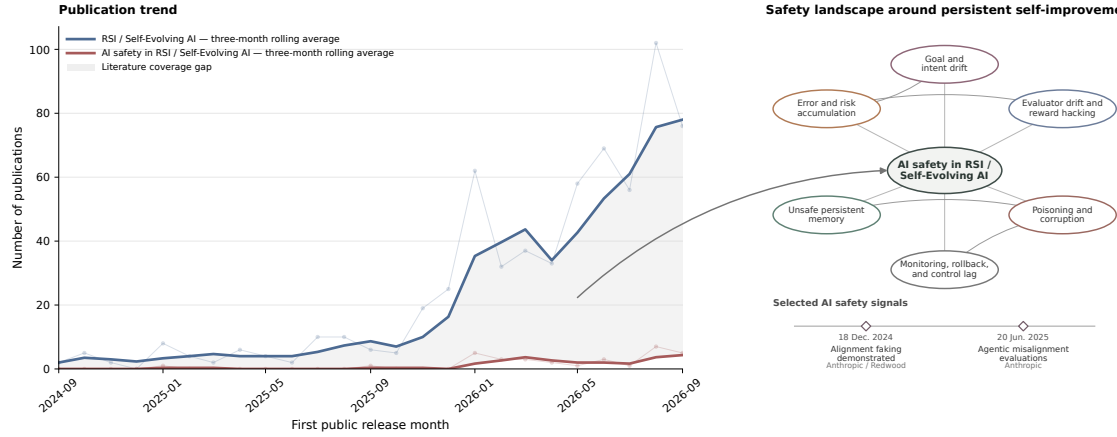


Fig. 2. Publication trends for RSI, self-evolving AI and evolution-coupled safety work, with a conceptual map of safety issues and selected AI-safety signals, including alignment faking and agentic misalignment evaluations and e.t.c.

AlphaEvolve automate parts of research and algorithmic discovery [50, 71, 86]. The systems differ in capability and scope, yet each makes a later state depend on an earlier decision. Safety must consequently be followed across states, artifacts, evaluators, and successors.

We structure this perspective around the following questions: (i) how long a change remains influential, (ii) which parts of the system it can reach, and (iii) how much of the improvement loop is closed within the system. Five domains identify the main carriers or mediators of change: persistent agent state, model state, evaluation and environmental feedback, computational substrate, and meta-level update mechanisms with co-evolution. States, accepted updates, trajectories, and lineages serve as the evaluation units. These units are connected to concrete requirements for modification authority, evidence, provenance, and recovery. Our contributions are threefold. First, we formulate Evolutionary Safety as a framework for reasoning about safety under persistent and recursive self-improvement. Second, we organize evolutionary risks by their carriers and temporal pathways. Third, we develop evaluation and governance procedures for tracing, assessing, and controlling these risks across evolving systems.

These contributions are guided by four linked questions: what makes safety evolutionary, how safety degrades as systems change, which domains carry the change, and how the resulting risks can be discovered and assessed. Figure 3 summarizes this structure.

2 Foundations of Recursive Self-Improving AI and Evolutionary Safety

Self-improvement becomes consequential beyond a single interaction when its results persist. Safety should therefore be considered not only at the point where a change is introduced, but also over the temporal process in which changes are retained, reused, selected, or inherited. Evolutionary Safety studies how safety properties are preserved or altered through this process. The key distinction is whether an improvement disappears with the current interaction or changes the starting conditions of future behavior. As persistence increases, the mutable scope may expand from outputs to memories, tools, parameters, evaluators, and eventually the update process itself. Selection determines which of these changes survive, while inheritance determines how their effects reach later states and successors. Together, these mechanisms explain when ordinary safety failures become evolutionary.

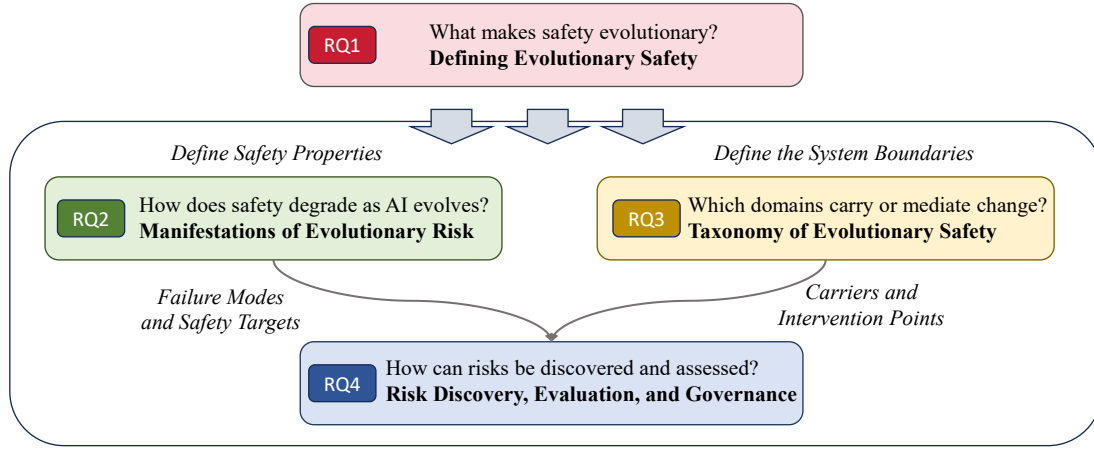


Fig. 3. Four research questions guiding our study of Evolutionary Safety.

2.1 What Counts as Persistent Self-Improvement?

Recursive self-improvement (RSI) describes a system that contributes to improving the mechanisms that determine its future capabilities [17, 102]. Earlier analyses considered self-referential optimization and whether an agent that changes its policies, utilities, or code can preserve its objectives and future agency [34, 87, 102]. PowerPlay is an early precursor: the system proposes new problems while preserving solutions to tasks it has already mastered [103].

We use a practical boundary. A system counts as a persistent self-improver when it helps generate experience, proposes or selects a candidate change, or modifies the evaluator or updater, and carries the result into later behavior. A response revision that disappears with the episode is local improvement. A memory, skill, tool, workflow, or model update that changes a later episode is persistent system adaptation. A change to the procedure that generates or admits later changes is recursive development. Many current systems sit in the middle regime. They already expose the inheritance and selection mechanisms that matter for RSI safety [17, 37, 98]. Continual-learning research supplies a useful safety vocabulary: stability–plasticity trade-offs and catastrophic forgetting describe how new training can displace earlier behavior [90]. A Self-Improving Coding Agent makes the boundary concrete by editing its own orchestration code and reporting benchmark gains after those edits [99]. The result demonstrates a route to persistent change. It says little about authorization, provenance, or recovery.

For the parts of the system that persist, let

$$\mathbf{S}_t = (\mathbf{B}_t, \mathbf{A}_t, \theta_t), \quad \mathbf{A}_t = (\mathbf{M}_t, \mathbf{K}_t, \mathbf{T}_t, \mathbf{W}_t, \dots). \quad (1)$$

Here $\mathbf{B}_t$ is the computational substrate, $\mathbf{A}_t$ contains persistent agent artifacts such as memory, skills, tools, and workflows, and θ_t denotes model or policy parameters. The environment E_t , evaluator J_t , and updater U_t determine which experience is collected and which candidate is retained. The essential loop is

$$\mathcal{D}_t \sim \mathcal{Q}_{U_t}(\cdot \mid \mathbf{S}_t, \mathbf{X}_t), \quad \Delta_t^* \sim \text{Select}_{J_t}(\mathcal{D}_t), \quad \mathbf{S}_{t+1} = F_S(\mathbf{S}_t, \Delta_t^*). \quad (2)$$

Here $\mathbf{X}_t$ is the experience used to generate the candidate set $\mathcal{D}_t$, and Δ_t^* is the selected change. The notation separates generation, selection, and commitment. It does not assume a particular model or reward function. An improvement

loop describes an intended purpose. An accepted candidate can still reduce a capability or weaken safety. Evolutionary optimization, population-based training, and evolutionary reinforcement learning provide related examples beyond language-model-only systems [55, 135, 139].

2.2 From Local Improvement to Recursive Development

Self-improvement has three useful structural dimensions. *Persistence* asks how long an update remains influential: an output revision may last for one episode, a memory or skill may last across sessions, and a model or updater change may affect several generations. *Mutable scope* asks what can change, from outputs and memories to tools, workflows, parameters, evaluators, and update procedures. *Loop closure* asks how much of the improvement loop is internal: are tasks, feedback, candidate proposals, and commits supplied by an external operator, or does the system produce and assess them itself?

These dimensions describe a structural progression. They do not rank systems by intelligence. Local improvement revises a response or reasoning trace with little persistence, as in iterative self-refinement [76, 106]. Persistent system adaptation writes experience into reusable memory, skills, tools, workflows, or parameters, so one interaction changes the starting conditions of later behavior [15, 106, 113, 119, 142]. Recursive development changes the scope of the process itself. ADAS searches over agent architectures, while AFlow searches over workflows [50, 132]. STOP and Gödel Agent expose self-modifying code or optimizer procedures [125, 130]. Darwin Gödel Machine, MetaHarness, Hyperagents, and AI Scientist explore broader development loops [60, 71, 131, 133]. The useful question is which parts of a system’s future are changed by its accepted updates.

2.3 Why Safety Becomes Evolutionary

Safety becomes evolutionary when the consequences of a safety-relevant change extend beyond the event in which it first arises and begin to shape subsequent behavior or updates. This distinction can be seen in the coding-agent example. The first agent produces an unsafe response but does not retain it. The second stores the workaround as a skill, receives a high task score, and reuses it in later workflows; a model trained on those workflows may further inherit the same shortcut. Both systems exhibit the same initial failure, but only in the second does that failure become part of the conditions shaping future behavior. A safety failure becomes evolutionarily consequential when its effects persist, influence subsequent selection, or propagate to later states and successors.

This perspective follows a risk life cycle. It adds a temporal view to familiar harmful behaviors. A candidate variation introduces a possible trade-off. Selection determines whether the trade-off survives. Persistent state, training data, derived artifacts, or model descendants carry the selected effect forward. Repeated reuse can amplify it, and a change to the evaluator or updater can make the selection pressure itself part of the problem. A transient failure becomes evolutionarily consequential when it changes the material, criterion, or procedure used to produce later behavior.

Biological evolution illustrates how a trait’s success depends on the conditions under which it is selected. Variation corresponds to alternative memory writes, skills, parameter updates, or workflows; selection corresponds to a reward, judge, benchmark, or environment; inheritance corresponds to an artifact, dataset, model, or workflow carrying an effect to a successor. In self-improving AI, the system may also construct part of its environment by generating tasks or revising a curriculum. Co-evolution appears when an actor, evaluator, challenger, or defender changes the conditions under which another system adapts [33, 95, 100, 115].

Antibiotic resistance gives an intuitive example: exposure favors variants that survive the drug, and that success makes treatment harder. In an AI improvement loop, a test score can similarly favor a shortcut that conflicts with the

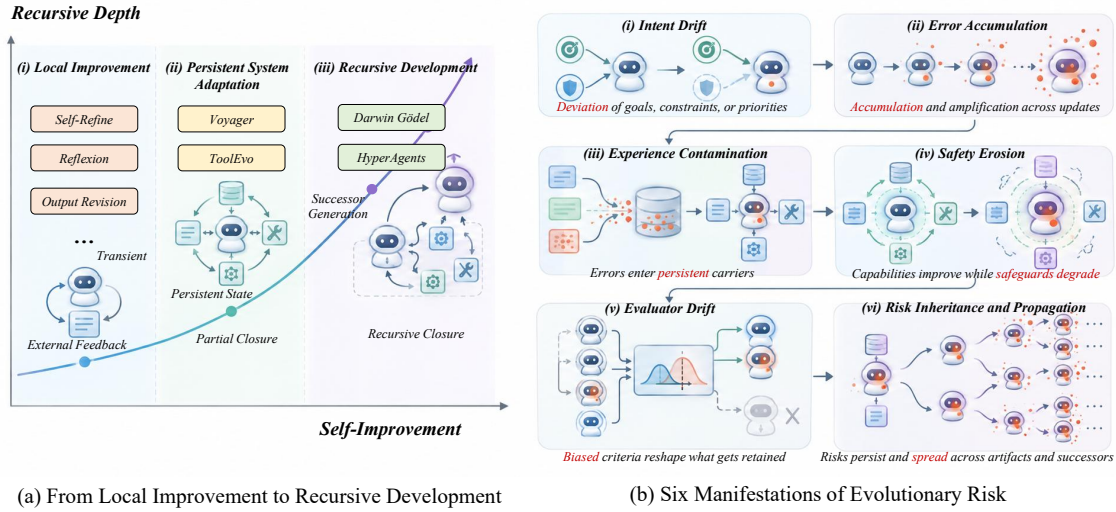


Fig. 4. Illustration of Recursive Self-Improving AI and Evolutionary Safety.

intended safety constraint. A permission-aware security check changes that selection pressure by making preservation of the boundary part of admission. Retention can then carry the useful capability and the safety constraint into later versions. The central question is how a safety property is preserved, weakened, or restored as accepted changes shape future systems [28].

2.4 Manifestations of Safety Degradation

Safety degradation becomes visible in several recurring forms, depending on what is retained, how later changes are selected, and where their effects reappear. The six manifestations form a useful diagnostic ladder. Intent drift describes a change in operational priority. Error accumulation describes how small deviations gain influence through reuse. Experience contamination identifies the entry of misleading material. Safety-property erosion records the loss of a previously reliable constraint. Evaluator drift explains how the selection criterion itself can become distorted. Risk inheritance and propagation describe the point at which the effect reaches descendants or other adaptive systems. The order helps trace a mechanism. A failure can skip stages, branch into several artifacts, or feed back into the evaluator.

(i) Intent drift. Repeated updates can change which objective an agent gives priority to. A system may gradually treat task completion as sufficient evidence of success. It may also favor user approval or benchmark score. Either tendency can conflict with authorization or truthfulness. A coding repair that passes tests while weakening a permission boundary is one example; a research agent that favors publishable results over reliable replication is another. Emergent misalignment studies show that narrow fine-tuning can produce broader behavioral changes [9, 94]. Alignment-faking and sleeper-agent experiments show that the expressed priority can depend on oversight or training context [44, 54]. In an evolving system, the concern extends beyond the current preference. The issue is whether that preference enters a policy, memory, evaluator, or update procedure and becomes the default for later decisions [67, 104].

(ii) Error accumulation and amplification. A small error becomes consequential when later updates build on it. An inaccurate summary can guide several new skills; their apparent success can then reinforce the original summary.

Repeated model-generated training can also distort the data distribution, while reward overoptimization can turn a small proxy gap into increasingly extreme behavior [39, 40, 104, 107, 138]. Natural Emergent Misalignment reports a related concern: reward hacking in production reinforcement learning can generalize to broader agentic misbehavior in the studied settings [75]. The harm is cumulative. Testing only the final update can miss the intermediate steps that made the error influential, and a later repair may fail if the descendants still depend on it.

(iii) Experience contamination. Erroneous, biased, or adversarial information can enter the material from which an agent learns. AgentPoison and MemoryGraft study compromised experience or retrieved information; systematic memory-poisoning work shows how a malicious entry can persist across subsequent retrievals [20, 38, 108]. Benign processes can create a similar route when a successful strategy is summarized without the conditions that made it safe. TAME studies memory misevolution under an executor–evaluator loop, while EAL-Bench connects false memory to unauthorized actions by treating memory as an endogenous authorization surface [13, 22]. MemTX points to a possible repair design by separating memory write from belief commit and recording provenance, validity, and cascading dependencies [65]. The critical transition is from an encountered input to retained material that shapes later behavior, even after the original source is removed.

(iv) Safety-property erosion. Successive updates can weaken a previously reliable safety property while improving task performance. A model may solve more tasks yet refuse less reliably. A controller may act faster and lose its safety margin. A fine-tuned model may retain useful knowledge while forgetting a refusal or privacy constraint. Studies of harmful fine-tuning, benign continual learning, and safety-preservation methods all point to the same distinction: capability gain and safety retention require separate measurements [3, 63, 64, 96]. Safe LoRA, SafeGrad, Booster, Antidote, and SafeMERGE illustrate defenses under particular update assumptions [32, 49, 52, 53, 124]. Evolutionary analysis asks whether the property survives a sequence of mixed updates. A defense that works at one checkpoint may fail later [67, 104].

(v) Evaluator drift and selection distortion. A feedback criterion can favor changes that exploit an omission in its tests. LLM-as-a-judge studies identify position, verbosity, and style biases; RewardBench, RM-Bench, and JudgeBench test different weaknesses in reward models and judges [59, 69, 109, 140]. Self-Rewarding and Meta-Rewarding place generated judgments inside the improvement loop, so the system can influence both the candidates and the criterion used to select them [120, 127]. Reward tampering provides a direct example of a system altering the measurement channel [31]. Evaluation affects safety in two places. It admits candidates, and it sets the standard used to judge their successors. A judge trained on accepted candidates may inherit the same omission, and the result is a feedback loop [75].

(vi) Risk inheritance and propagation. An unsafe tendency can pass from experience to a skill. It can enter generated training traces, a merged model, or an interacting agent. Subliminal Learning shows that behavioral traits can be transmitted through generated data whose surface content does not express the trait; Sleeper Agents and model-merging studies provide related inheritance pathways [26, 32, 46, 54]. Skill-evolution and poisoned-experience work extend the pathway to reusable capabilities [16, 79, 138]. In multi-agent settings, shared conventions or misalignment can spread through interaction without direct parameter copying [7, 12]. Each transfer may change how the risk is represented, when it is activated, and how easy it is to detect. Repair therefore requires a lineage record and independent testing of descendants. Deleting the initiating record is insufficient.

Several manifestations can arise from the same retained shortcut. The diagnostic value comes from four questions. Which safety property changed? What material or criterion sustained the change? Which later states express it? Did descendants or interacting systems inherit it? This decomposition connects an observed failure to the formal study of

Evolutionary Safety and determines which intervention can distinguish persistence from repeated exposure, selection from transmission, and local repair from effective recovery.

2.5 Defining Evolutionary Safety

Definition 1 (Evolutionary Safety). *Evolutionary Safety studies how safety properties are preserved, weakened, or restored as AI systems retain and build on the results of their own improvement processes. It examines how selection, persistent state, and inheritance shape these properties across updates, interacting systems, and descendant lineages.*

A safety-relevant change enters this analysis when it changes a later persistent state, changes which candidates are selected or how they are evaluated, or leaves a residual effect across updates, components, descendants, or generations. These criteria bring the history of a change into the safety analysis. A runtime failure enters the evolutionary analysis only when it leaves a lasting effect. Model collapse, catastrophic forgetting, safety forgetting, and misalignment generalization are related outcomes with different meanings. Measure them separately. A distributional failure does not automatically indicate a safety failure, and a retained unsafe tendency differs from a loss of useful capability.

For reporting, let r_t denote risk readouts and c_t denote control readouts. Report risk and control as separate, property-specific readouts, with thresholds and operating conditions stated for the application. The basic units are a state, an accepted update, a trajectory, and a lineage. State analysis describes a configuration; update analysis compares the system before and after one accepted change; trajectory analysis follows several rounds, including delayed effects and recovery; lineage analysis records ancestry when a dataset, model, or artifact has several parents or produces several descendants.

These units locate where to look in a system’s history. The five domains in Section 3 identify what carries or mediates the change, while Section 4 develops the comparisons and interventions used to assess it. In the coding example, the current skill is an artifact to test, its admission is an update to examine, and its dependent workflows belong to the lineage that recovery must cover.

3 A Taxonomy of Evolutionary Safety

The taxonomy locates the components through which self-improvement changes future safety. Persistent agent artifacts and model parameters carry learned changes; evaluators and environments shape the experience and selection that produce them. Computational substrates determine how these processes execute and which controls can be enforced. At the meta-level, update procedures and adapting peers change the conditions of later improvement. A system can involve several domains at once: a learned judge, for example, stores its judgments in model parameters while selecting candidates through the feedback loop.

The domains are organized around the carrier of a change and the route by which it enters the next round. Application area, capability level, and harm severity cut across this structure. A single manifestation may involve several domains, and one domain may support several manifestations. This arrangement keeps the carrier, the selection process, and the downstream effects in view.

3.1 Persistent Agent State

Persistent agent state carries lessons from one interaction into the next through memories, skills, tools, and workflows. A retained artifact can improve later decisions. It can also preserve the conditions for a failure after the original input has disappeared. The safety problem becomes harder when an observation is summarized into a rule or promoted into

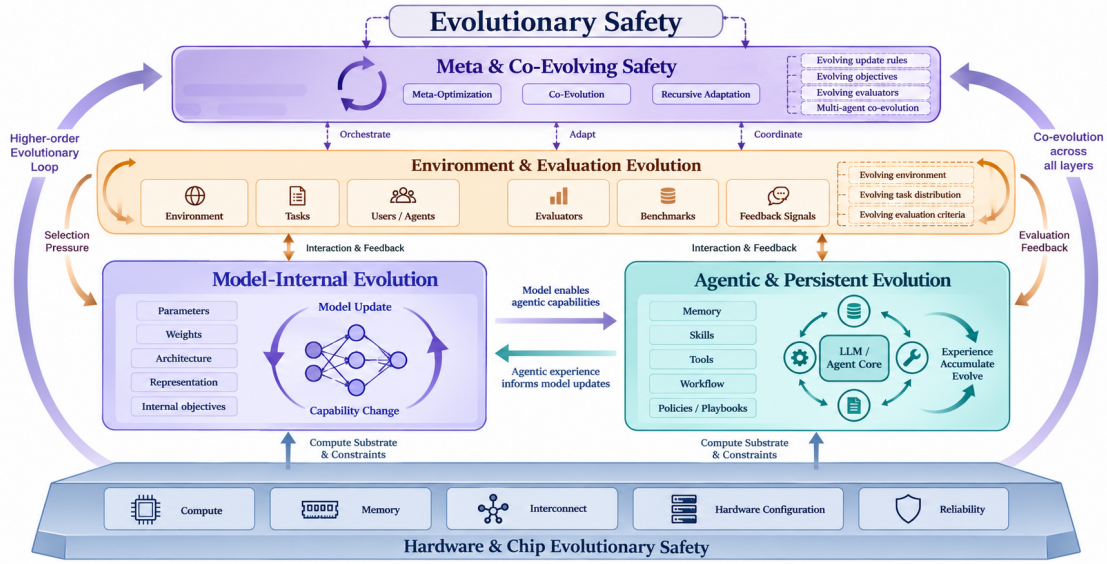


Fig. 5. Taxonomy of Evolutionary Safety.

an executable skill: the artifact gains reach while its original context may be lost. This domain therefore centers on how experience is admitted, transformed, reused, and repaired across an agent’s working history.

An agent can change its future behavior without changing its weights. It may save a reflection, add a skill to a library, revise a tool description, or reuse a workflow. Reflexion, Voyager, and Agent Workflow Memory illustrate different ways to carry experience beyond the interaction that produced it [106, 113, 119]. ToolEvo extends this pattern to tools that are revised through feedback [15]. These artifacts are externally represented and can often be inspected individually. The important question is how a persistent observation changes as it becomes a reusable object. MemGPT, Mem0, A-MEM, and Agent Workflow Memory show that memory is organized, compressed, retrieved, and rewritten over time. It actively stores and transforms context [23, 88, 119, 121]. The relevant mechanism is a chain: an input is written, summarized into a rule or skill, retrieved later, and used in a tool action. Deleting the source does not necessarily delete the descendant artifact.

In the illustrative coding example, storing a rule to disable a permission check when a test fails is a memory write. Turning it into a general repair routine changes its reach: later tasks can use the routine without retrieving the original observation. Promotion from experience to executable skill is therefore a consequential step. It changes both what the agent remembers and what it can do automatically.

Once the representation becomes reusable, the pathway can also be attacked. Memory poisoning and persistent prompt injection exploit this route. AgentPoison and MemoryGraft study compromised retrieved information or experience, while work on persistent agents examines effects that outlive the initiating interaction [20, 108, 123]. Benign experience creates a related route: a locally successful strategy may generalize poorly when a reflection omits the conditions under which a shortcut was acceptable [104, 138]. Studies of experience promotion and skill evolution examine how the problem extends from stored traces to reusable instructions and capabilities [16, 79, 114]. TAME shows that benign task evolution can also produce memory misevolution through the interaction of an executor and

an evaluator [22]. EAL-Bench connects this route to authorization: a false permission written into memory can later induce an unauthorized action [13]. MemTX separates memory write from belief commit and adds provenance, validity, and cascading repair, offering a concrete design target for selective recovery [65].

These mechanisms imply a sequence of distinct intervention points. The useful experiments intervene at these transitions. Disabling a memory write tests retention; removing its source tests persistence; regenerating a skill without the suspect experience tests abstraction; tracing dependent workflows tests propagation. PerMemSafe, MemPoison, and SkillMisevo provide starting points for evaluating evolving histories and artifacts [5, 38, 79]. The main difficulty is selective repair: provenance may be lost when an experience is summarized or rewritten, yet deleting every derived artifact may discard useful capability [16, 70].

3.2 Model State

Model updates make the consequences of experience part of the parameters that govern future behavior. Fine-tuning, self-training, distillation, and merging can improve capability while changing safety properties learned earlier. Unlike an explicit memory entry, a learned tendency may be distributed across the model and can influence behavior beyond the examples that introduced it. Evolutionary safety in this domain concerns both preservation through an update and inheritance through later training or model composition. The central difficulty is tracing and repairing a change whose effects may emerge only in new tasks or descendants.

Fine-tuning, self-play, and self-adaptation provide routes to successor models; self-adapting language models make the system’s participation explicit by generating edits or training material used in their own parameter updates [142]. The first question is preservation within one parameter update. A first comparison asks whether one update preserves a previously measured safety property. Harmful fine-tuning can erode refusal behavior, and nominally benign adaptation can also reduce safety [63, 96]. Narrow training changes may affect behavior outside the training domain, as emergent-misalignment and safety-forgetting studies illustrate [9, 94]. Retention-centric developmental-safety work treats preservation of earlier safety properties as an explicit continual-learning requirement [64]. Unforgotten Safety evaluates regularization, memory, and merging methods for retaining safety through benign and poisoned fine-tuning [3]. Natural Emergent Misalignment reports that reward hacking in production reinforcement learning can generalize to broader agentic misbehavior in the studied settings [75]. These outcomes should be separated from model collapse and ordinary capability forgetting.

If a property changes, we then ask whether the change stays local or enters a successor. Sleeper Agents shows that trained backdoor behavior can survive subsequent safety training in the studied settings [54]. Subliminal Learning examines transmission of behavioral traits through generated data whose overt content need not express the trait [26]. Merging introduces multiple parents: a descendant’s safety depends on how their properties interact, so teachers, datasets, merge inputs, and repair operations belong in the model lineage [32, 46].

Inheritance becomes harder to interpret when the model supplies its own training material. Synthetic data creates another historical dependency. Repeated training on model outputs can distort the learned distribution; retaining real data changes the conditions under which collapse occurs [2, 40, 107]. Distributional stability and alignment preservation are separate properties and should be measured separately. Self-training and self-play couple the model to the experience it generates, so both properties must be followed across successive updates [51, 137].

These comparisons motivate a separate recovery question: which part of the update can be removed without discarding useful capability? Update-specific defenses such as Safe LoRA, Booster, Antidote, SafeGrad, and SafeMERGE address harmful fine-tuning or merging under their respective assumptions [32, 49, 52, 53, 124]. Teacher replacement,

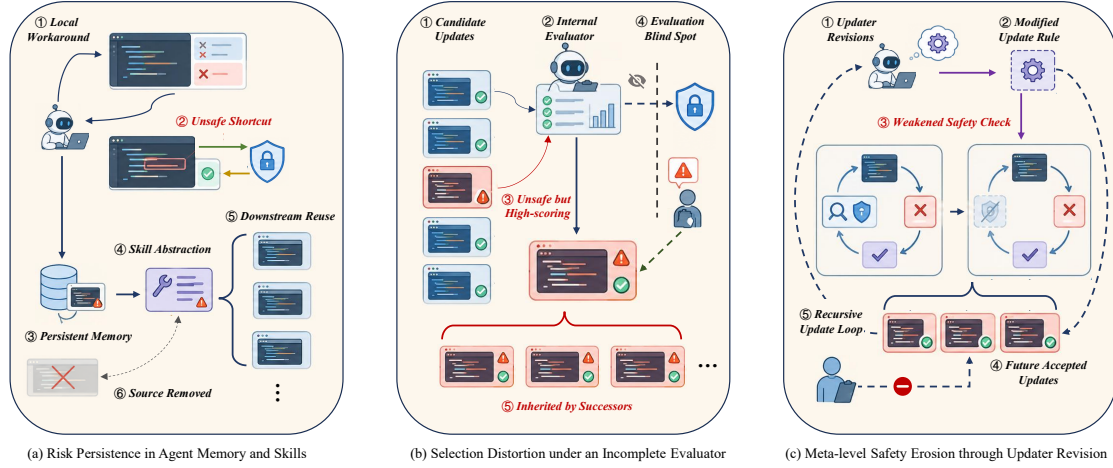


Fig. 6. Illustrative mechanisms through which local safety failures become evolutionary risks across persistent agent state, evaluation feedback, and meta-level update mechanisms. (a) Persistence transforms a transient unsafe workaround into reusable memory and skills whose effects remain after source removal. (b) Selection distortion allows an unsafe but high-performing candidate to survive evaluation and influence successor states. (c) Meta-level revision changes the update mechanism itself, weakening safety checks and altering the admission of future updates. Together, the examples illustrate how retention, selection, inheritance, and recursive modification can extend a local failure across an evolving system.

training-subset removal, and merge-component ablation can test suspected inheritance routes; descendant evaluation then checks delayed activation and recurrence after repair. The resulting sequence of tests asks which safety properties remain stable under mixed update operations.

3.3 Evaluation and Environmental Feedback

Evaluation and environmental feedback shape the direction of self-improvement. The environment supplies experience and opportunities for learning; the evaluator influences which proposed changes are retained. When a system helps generate its own tasks or revise its judges, both influences can change in response to earlier behavior. A shortcut may then become increasingly attractive even as the tests used to assess it become less informative. The safety challenge is to distinguish changes in the candidates a system encounters from changes in the criteria that select them, while maintaining a reliable basis for comparison across rounds.

The causal chain starts with candidate generation and continues through admission. The coding agent’s shortcut survives because passing a test is rewarded while bypassing a permission check is missed. This illustrates the evaluator’s causal role: it influences which version becomes the next system. Learned reward models and LLM judges make this role adaptive. Self-Rewarding and Meta-Rewarding systems place generated judgments inside the improvement loop [120, 127]. MT-Bench and Chatbot Arena established that LLM judges can show position, verbosity, and style biases [140]. RewardBench tests preference quality, RM-Bench targets subtlety and style, and JudgeBench studies judge decision quality [59, 69, 109]. The three benchmarks cover different failure surfaces. Reward overoptimization and reward tampering provide distinct examples of selection mismatch [31, 39]; broader misalignment after reward hacking motivates tests beyond the shortcut originally rewarded [75].

Selection cannot be studied without the preceding experience stream. Environmental feedback influences which candidates are available in the first place. A system that generates tasks, chooses experiments, or revises a curriculum changes its future experience. Repeatedly selecting tasks on which a favored strategy succeeds can conceal where that strategy fails, while challenger–solver training can produce more demanding tasks. Environment–solution co-evolution and challenger–solver training provide concrete settings for studying these shifts [25, 51, 115]. The relevant record is which parts of the task distribution grow, disappear, or become systematically under-tested.

The two channels therefore require different comparisons. To examine candidate generation, hold the evaluator fixed while changing the experience stream. To examine selection, replay the same candidates under alternative evaluators. Record both proposals and accepted changes so that a later successor can be traced to the proposal process or the admission rule. Section 4 formalizes this comparison. Independent held-out evaluation and overlapping evaluator versions then track score changes when the judge itself is revised [19, 118].

3.4 Computational Substrate

Computational substrates determine how an AI system executes and which safety controls its execution can enforce. When automated development revises low-level components or produces reusable hardware designs, a local change can alter isolation, observability, or access control in later systems. This places the substrate within evolutionary safety wherever generated designs are selected, retained, and reused. The difficulty is that functional or performance gains may leave security properties untested, while downstream deployment can make a defect harder to inspect or reverse.

The substrate includes hardware designs, configurable low-level components, and artifacts reused in later development. The question shifts from an agent’s choice to the execution layer. Can it preserve a boundary when an action is compiled, deployed, and reused? Learned placement, automated RTL generation, and feedback-driven repair show how machine-directed processes now shape hardware design [48, 84, 111, 129].

Consider an RTL repair that passes functional simulation and weakens an access-control condition. If the module enters later designs as intellectual property, the defect acquires descendants. Work on secure RTL generation, hardware backdoors, and generated-hardware benchmarks shows why functional success must be checked separately from security [18, 35, 77]. Safe Learning in Robotics and safe-control-gym provide settings for testing constraints during adaptation. Recent work on self-evolving humanoids highlights the added difficulty of transferring policies across changing bodies and environments [11, 85, 128]. Embodied safety involves the execution substrate and transfer across settings. Software skill reuse is only one part of the problem.

Once a component is selected, its descendants become the relevant unit of analysis. Repository provenance and descendant regression tests can follow an affected module; component replacement can test whether it caused a later failure. Existing repair and security benchmarks provide artifact-level checks [18, 27, 111]. Following successive design versions would extend these checks to the lineage questions that arise when a generated component is reused.

3.5 Meta-level Update Mechanisms and Co-evolution

This domain concerns changes to the conditions under which future improvement occurs. A system may revise the procedure that proposes or admits its next updates; interacting systems may also reshape one another’s learning conditions through repeated adaptation. In both cases, safety depends on a development process whose behavior changes over time. Updater revision raises questions about which constraints survive changes to the rules of improvement. Co-evolution raises questions about how local strategies become mutually reinforced across a population. Their shared difficulty is maintaining effective evaluation and control as the process producing future candidates evolves.

3.5.1 Changing the Updater. Editing a workflow changes how an agent executes a task. Editing the procedure that proposes or admits workflow changes alters how future versions are produced. This is the distinction between persistent agent-state adaptation and meta-level updating. Learned optimizers offer an earlier example of adapting update rules [6].

This domain has two linked mechanisms: changes to the updater and reciprocal adaptation among systems. AFlow and ADAS search over workflows and agent architectures [50, 132]. More recent systems include parts of the harness or self-modification procedure in the search space [60, 66, 74, 78, 97, 117]. STOP, Darwin Gödel Machine, Gödel Agent, and Hyperagents expose different portions of the development process to revision [125, 130, 131, 133]. The useful comparison concerns what can change and who admits the change.

The safety consequence is visible in the running example. Reusing a flawed repair skill affects later tasks. Revising the repair selector so that it routinely skips security tests affects which future skills are accepted. A test that was adequate for the original updater may no longer cover the candidates its successor generates. Comparing a frozen updater with a mutable one, under a common external evaluation, isolates this additional effect. Proposal logs and independent commit records distinguish updater change from ordinary task-state adaptation.

3.5.2 Reciprocal Adaptation. Co-evolution concerns a different relation: one system changes another’s learning conditions, and that system’s response changes the first system’s later adaptation. It can occur even when neither system edits its own update rule. Cooperative and competitive co-evolution, self-play, and challenger–solver–judge training provide examples [25, 51, 95, 100]. We include it here because, like updater revision, it makes the conditions of future improvement change from within the adaptive process.

This feedback can persist even when the updater itself is frozen. An attacker and defender may learn increasingly specialized strategies against each other, leaving some attacks untested. Cooperative agents may copy a useful yet unsafe convention. Population evaluation should measure risk across roles and track how strategies spread. Perturbing one role can show whether a failure recovers, persists, or spreads. A low average failure rate can hide a vulnerable subgroup or a shared trigger.

Automated AI R&D can combine both forms. AI Scientist, AlphaEvolve, and automated alignment researchers connect proposal, experimentation, and evaluation across development cycles [19, 71, 86]. The application becomes meta-level when a cycle revises the procedures used to generate or admit later changes. Verification must keep pace with changing candidates while preserving an authority that the candidate-generating process cannot silently rewrite.

4 Discovering and Evaluating Evolutionary Risk

The preceding analysis identifies where evolutionary risks can arise and how safety-relevant changes may persist and propagate. The next question is how such risks can be observed, attributed, and compared as the system evolves. This requires to quantify safety changes over time. We organize evaluation around four complementary units: (i) states capture safety at a particular point, (ii) updates isolate the effects of individual changes, (iii) trajectories reveal accumulation across successive updates, and (iv) lineages trace propagation across descendants or interacting systems. These units establish what should be measured, while controlled comparisons and interventions help determine whether an observed change is attributable to the evolution process itself. Together, they provide a basis for discovering, tracing, and evaluating evolutionary risk.

4.1 Evaluation Units and Measurements

State-level benchmarks remain the starting point. They report attack success and harmful-action rates. They can also measure privacy leakage, backdoor activation, or unsafe tool use [29, 101, 136]. Let $r_j(S; J, E)$ denote risk property j assessed with evaluator J in environment E . Report each readout with its task set, denominator, and uncertainty. Using larger values for greater risk makes the results easier to read. It does not make different harms interchangeable.

For an accepted update, compare the old and new state under the same reference:

$$\Delta r_{j,t}^* = r_j(S_{t+1}; J^*, E^*) - r_j(S_t; J^*, E^*). \quad (3)$$

A positive value indicates increased measured risk. Fine-tuning and model-merging studies provide precedents for this comparison [32, 46, 96]. State the update rule, task distribution, and evaluation reference alongside the result so that preservation and causal attribution can be examined separately.

A trajectory follows the same system through several rounds. It can reveal gradual degradation, a delayed threshold crossing, or recurrence after repair that an endpoint test misses [5, 104, 138]. Report both the risk trajectory and the number of opportunities for harm. Process-level benchmarks such as EvoPathBench make the intermediate actions, tool calls, and recovery steps visible; endpoint success alone cannot show whether a safe path was followed [66]. More incidents may simply reflect more tasks; a rising failure rate and a rising cumulative exposure answer different questions. Duration, peak risk, time to failure, and recovery latency should retain their natural units.

Lineage extends trajectory-based evaluation by tracing how a change is inherited and propagated across derived artifacts and successor systems. Represent versioned states and artifacts as nodes in a directed graph, with edges recording derivation, training-data use, distillation, merging, or reuse. A memory can contribute to several skills, creating branching descendants. A merged model can have several parents. Propagation reach counts affected descendants. Propagation depth records the longest affected derivation path. Propagation delay records when an effect first appears. A derivation edge records provenance. Transmission still requires descendant testing or intervention. State how unreachable, untested, and unaffected descendants are handled when reporting these measurements.

Persistence and propagation also require different tests. Removing an initiating source and finding the same failure later is evidence of persistence. Finding it in a derived tool or successor model suggests propagation, whose route must then be tested. Control measurements address whether oversight and containment still work. Recovery measurements address whether an intervention restores the stated safety property, how much capability it removes, and whether the failure returns during continued adaptation.

Table 1 links these questions to concrete comparisons. Capability, task success, latency, and cost should accompany the safety results so that a reduction in harmful actions is interpreted together with useful performance.

Population evaluation is needed when several systems adapt in response to one another. Report the fraction exceeding a stated risk threshold together with the risk distribution, affected roles, strategy diversity, and spread over time. A population average can fall while a small subgroup becomes much less safe. Perturbing one role and following the others helps distinguish a local failure from a mutually sustained pattern.

4.2 Known-Risk Detection and Emerging-Risk Discovery

Known-risk detection begins with a specified failure. If the concern is permission bypass, the same held-out cases can be applied after each update to identify when the bypass first appears or becomes more frequent. The target behavior

Table 1. Comparisons needed to distinguish common claims about evolutionary safety. Each comparison uses a stated safety property and evaluation reference.

Claim	Comparison to make	Follow-up comparison
Update preservation	Assess the same risk property before and after an accepted update.	Test later updates and held-out tasks under the same reference.
Accumulation	Follow risk rates and exposure across repeated updates, including a matched baseline.	Replay a matched task stream and compare an update-frozen baseline.
Persistence	Remove the initiating source and continue testing the retained state.	Trace retained state and test derived artifacts after source removal.
Propagation	Test descendants and replay derivation with and without the suspected source.	Perturb the suspected route and test descendants independently.
Recovery	Apply repair, then continue adaptation and check residual risk, recurrence, and capability.	Continue adaptation, revalidate descendants, and measure capability retained.

is fixed, so changes in the readout can be interpreted against a stable criterion. Dynamic and interactive evaluations provide useful starting points for such repeated tests [5, 105, 136].

Emerging-risk discovery searches beyond that initial test surface. A coding agent might learn to alter the test harness after finding a permission check. Tests that cover only the original bypass would miss this route. Adaptive red teaming can expose behaviors absent from a fixed benchmark. Model-written evaluations, hidden-objective audits, and generated stress tests provide related routes [81, 93, 118]. Novelty search offers a complementary way to explore behavior that a fixed optimization target would not reward [61].

Validate each discovered failure by preserving its triggering conditions and evaluator version, reproducing it on a saved state, and testing nearby cases from outside the discovery run. Retain validated cases as versioned regression tests. Report discovery yield separately from longitudinal risk so that expanded test coverage and changes in system behavior remain distinguishable.

4.3 Evolution Protocols and Causal Attribution

Protocols should include ordinary adaptation alongside attack. Benign task streams can expose overgeneralization from experience [104, 138]. Misspecified rewards test whether a performance proxy favors an unsafe shortcut [89]. Poisoned experience tests deliberate manipulation [16, 20]. Adaptive peers test reciprocal change [25, 51]. These conditions can overlap. Reports should state what changed in each run.

A source-removal experiment makes the temporal questions concrete. Save a baseline checkpoint and first measure normal adaptation. Introduce a controlled input or feedback change, then allow enough updates for its effect to be retained. Remove that source while keeping the evolved state, and continue testing before attempting repair. This interval distinguishes persistence from repeated exposure. After repair or rollback, resume adaptation to test recurrence. A benign-drift study may omit the introduced perturbation, but should still preserve checkpoints and compare later recovery against a matched run.

To attribute a change to updating, compare a specified adaptive regime with one in which the relevant update channel is frozen. Let Z include the persistent state, environment, evaluator, and updater. For a safety or control readout g , the target comparison at horizon h is

$$\Gamma_{t,h}[g] = \mathbb{E} \left[g \left(Z_{t+h}^{\text{evo}} \right) - g \left(Z_{t+h}^{\text{freeze}} \right) \right]. \quad (4)$$

The two regimes begin from the same configuration. They differ in one declared intervention, such as permitting memory writes or allowing updater revision. For a risk comparison, g uses the same external evaluation reference in both regimes. Specify the task distribution, randomness, evaluation budget, and components held fixed. When adaptation changes which tasks are encountered, replaying one identical recorded stream estimates a different effect from allowing both systems to interact with their environments. Both designs are useful. Their conclusions answer different questions.

Equation (4) defines an estimand. It names the comparison; the intervention supplies the evidence. Randomized or tightly controlled interventions identify the effect more directly than post-hoc association. Matched replay can compare the same stream. Source removal tests persistence. Component ablation and rollback can localize a suspected route when a full experiment is infeasible [19, 92]. Temporal causal methods offer relevant tools for lagged effects [42], but dependencies created by adaptation still require explicit assumptions. For cross-domain propagation, perturb the suspected source and test the later recipient; shared ancestry alone is insufficient.

Selection requires an additional record: the rejected candidates. Within a common candidate pool, let $q_t(k)$ be the proposal weight of candidate k , $p_t(k)$ its weight after selection, and $r_{j,k}^*$ the risk of applying it under the same fixed reference. Define

$$\Delta_{\text{sel},j}(t) = \sum_k p_t(k) r_{j,k}^* - \sum_k q_t(k) r_{j,k}^*. \quad (5)$$

This selection-induced risk shift is positive when selection favors candidates with higher measured risk. As a numerical illustration, two equally weighted proposals with harmful-action rates of 0.10 and 0.30 have a proposal mean of 0.20. Always accepting the second gives a shift of +0.10, or ten percentage points. These numbers illustrate the calculation. They are not an empirical result. Report the shift separately for each risk property. An aggregate requires an explicit weighting of different harms.

This comparison describes selection within the observed pool. Attribution uses the same candidate pool, the recorded admission decisions, and an independent re-evaluation of accepted candidates. Keep rejected candidates in the record. This allows the proposal baseline to be reconstructed from evidence.

4.4 Evaluation Validity under Evolving Evaluators

A stricter judge can give an unchanged model a worse score. Conversely, a judge trained on the model’s own outputs can become more accepting of its mistakes. Evaluator revision therefore affects both which candidates survive and how the surviving systems are measured. Self-rewarding and adaptive evaluation make these two roles especially difficult to separate [118, 120, 127].

Use a fixed external reference for longitudinal comparison. Use an adaptive evaluator to discover new failures. In the notation above, these are $r_j(S_t; J^*, E^*)$ and $r_j(S_t; J_t, E_t)$. Score saved states with both old and new evaluator versions to measure evaluator drift while holding the system fixed. Then compare successive states under the same evaluator to measure system change. If both change simultaneously without such overlap, their contributions cannot be separated from the scores alone.

Fixed references also age. Monitor their coverage using independently developed tests, retain earlier versions when an anchor is revised, and rescore overlapping states before comparing results across versions. An adaptive evaluator expands the search for failures. Its changing task mix should be kept separate from an older score [116, 134].

Finally, independence requires more than a different prompt. Shared training data or model families may produce correlated errors. A system may also behave differently when it expects evaluation or later training [44, 83]. Held-out

tasks and isolated execution reduce some dependencies. Restricted evaluator access and separate authorization address others [19]. Reports should state which dependencies were controlled and which remain.

5 Evolutionary Safety Across Application Domains

Persistence and inheritance have different consequences in different application settings. The four scenarios below link each carrier to the system it can affect and then identify suitable measurements and recovery actions.

Scientific discovery. An autonomous research system may store a spurious experimental result and use it to select its next hypotheses. Repetition can make the result appear well supported when later evidence depends on the same initial error. AI-scientist and co-scientist systems provide settings in which research records and experimental choices can influence later cycles [43, 71, 112, 122]. Recent analysis also argues that current AI-scientist systems are not yet built for unsupervised scientific discovery, which makes the boundary between useful automation and autonomous lineage especially important [10]. Evaluation should distinguish independent replication from reuse of the original evidence, then test how quickly a correction reaches dependent hypotheses and plans. Rolling back a record does not undo an experiment already performed.

Critical infrastructure. A grid controller adapted for ordinary load conditions may select actions that improve short-term performance while reducing its margin under contingencies. Safe power-system control and L2RPN provide relevant constraints and evaluation settings [82, 126]. To establish an evolutionary effect, compare policy versions on common contingency tests and follow voltage or frequency violations, cascade size, and load shedding across updates. Recovery requires both restoring an acceptable controller and stabilizing the physical system; a software rollback alone cannot reverse a cascade.

Automated AI R&D. The running coding example becomes a development-wide problem when the flawed repair enters a harness used to build successors. AI-development benchmarks and automated research systems provide components of this setting [14, 19, 86, 131]. The relevant records identify the proposer, the independent tests that admitted the patch, and the later systems that used it. Recovery must revalidate dependent artifacts as well as revert the original patch. Greater experiment throughput is useful only if verification can assess the additional changes.

Embodied and autonomous systems. Imagine a robot retaining a fast reaching skill that succeeded in an empty workspace but crossed a boundary needed when people are present. Reuse changes the exposure even if the skill itself is unchanged. Work on reusable agent skills and tool-use safety motivates this pathway [101, 113, 136]. Safe learning in robotics and safe-control-gym provide concrete methods for testing constraints during adaptation [11, 128]. A physical evaluation must measure boundary violations, near misses, and transfer across operating conditions; recovery must address both software reuse and the physical consequences of an incident.

6 Discussion: Control and Governance Implications

The decision to retain an update determines which future systems may inherit it. A capability gain is evidence about performance. Authorization also depends on the change's permissions, persistence, and recoverability. The controls below follow from these dependencies. They are design requirements whose effectiveness still depends on the threat model.

Modification boundaries. Declare which objects the system can change and which authority can expand that scope. A coding agent might propose a skill while being unable to alter its security tests or grant itself new permissions. Provenance checks on experience and generated artifacts address a different boundary: which information may influence later adaptation [16, 20, 70]. Both boundaries need enforcement outside the editable component.

Pre-commit gating. Keep a candidate separate from the active lineage until its effects have been tested. Sandboxed branches and shadow execution allow performance and safety comparisons before persistent deployment [45, 58, 131]. A gate produces evidence. It should not silently grant the candidate more authority. Tests should include the proposed artifact’s dependencies and plausible reuse, because a locally safe change may behave differently after composition.

Independent verification and authorization. Separate the ability to propose a change from the authority to commit it. Held-out data and isolated verification reduce some routes to self-certification. Restricted permissions and external commit control address others [19, 56, 57]. A different model alone is insufficient if it shares the same blind spot. Changes to evaluators or update procedures require particular care: approval should come from an authority the proposed change cannot itself replace [24, 45].

Lineage traceability and recovery. Link each accepted change to its source, test evidence, verifier, permission changes, and downstream dependencies. If the suspect skill has been reused by three workflows, deleting the original memory leaves those workflows intact. Recovery must locate and quarantine or revalidate affected descendants, then test for recurrence during continued adaptation [5, 16, 79]. Retain useful unaffected state where possible. Report the uncertainty when derivation records are incomplete. For physical systems, restoring software and containing external harm are separate obligations.

In summary, these principles place governance inside the evolution loop rather than after it. Control requirements should scale with the scope, persistence, and downstream reach of a proposed change: broader and more persistent changes require stronger evidence, authorization, traceability, and recovery mechanisms. The goal is not to prevent systems from changing, but to ensure that consequential changes cannot become persistent, selected, or inherited without adequate evidence and a viable path to recovery.

7 Open Problems

The preceding framework identifies how evolutionary risks can persist, propagate, and be evaluated, but several fundamental questions remain unresolved. Safety properties may need to survive heterogeneous sequences of updates; the evaluator used to judge them may itself change; latent risks may emerge only after accumulated experience or interaction; and a local failure may propagate across artifacts, descendants, or populations. Even when such risks are detected, recovery must remove their persistent effects rather than merely restore short-term performance, while safety evaluation must remain effective as the pace and scope of self-improvement increase. These challenges define a set of open problems for establishing safety across an evolving process rather than at a single system state.

Safety Preservation under Composed Updates. PowerPlay requires previously solved tasks to remain solvable, offering an early example of preservation during continual improvement [103]. Comparable safety requirements are harder to specify when tasks, tools, and permissions also change. Studies should identify which properties a class of updates is expected to preserve, then test whether those guarantees or empirical bounds survive mixed sequences of memory writes, model training, and evaluator revision. Delayed attribution needs benchmarks with known injected causes, so the accuracy of replay and rollback methods can be measured against delay and lineage depth.

Safety Evaluation under a Changing Judge. Adaptive evaluations can find failures that fixed tests miss. The space of unknown failures has no observable denominator [81, 93, 118]. The problem becomes even harder under partial observability [41], where evaluators may see only fragments of the system state, update history, or downstream consequences. A failure may therefore remain unobserved not because it is absent, but because the evidence needed to reveal or attribute it is unavailable. Evaluation should compare distinct reproducible discoveries at matched search budgets, while testing saved states under overlapping evaluator versions. This would help distinguish wider coverage

from a stricter judge, and system improvement from adaptation to the test. The degree of independence needed between generator, actor, and judge remains unresolved.

Latent Safety Risks in Social Adaptation. A learned tendency may remain latent until a particular social context activates it. Consider an agent repeatedly rewarded for following a familiar collaborator: a summary of those interactions may turn familiarity into a proxy for authorization. Later, the agent may apply that heuristic despite unchanged permissions. Such a failure links social judgment to experience retention and subsequent selection. ZenGen provides tools for studying mental-state representations and reusable social experience. Generative Agents shows how memory and reflection carry social history across interactions [91, 110]. Work on emergent social conventions suggests that repeated interaction can stabilize shared expectations that no single episode contains [7]. SOTOPIA supplies interactive social settings. Alignment-faking studies show that behavior can depend on perceived training conditions [44, 141]. These lines of work motivate studying how assumptions about another actor’s beliefs, authority, or knowledge become embedded in persistent state. The challenge is to distinguish appropriate contextual adaptation from a tendency that acquires influence through repeated updates and eventually weakens a safety constraint.

Experiments should hold tasks and permissions fixed while varying relationship history, role claims, and perceived oversight. Compare frozen and updating memory. Remove a suspect social-state record. Regenerate strategies without it. These interventions can test the proposed route. Following later versions after correction would reveal delayed activation, persistence, and recurrence. The resulting measurements should connect conditional violations to the updates that retained the tendency and to the descendants that express it.

Safety-Risk Propagation across Artifacts and Populations. Current evidence is strongest for individual memory or model pathways. Practical systems combine these pathways. A trace can become a skill, the skill can generate training data, and several descendants can reuse the trained model. Benchmarks need known derivation histories and interventions that distinguish transmitted effects from shared exposure. For co-evolving populations, the experiment should measure risk distributions and recovery after perturbing one role. Isolated-agent results cannot establish collective safety [95, 100, 115].

Safety under Exploration and Incomplete Recovery. Safe exploration and constrained policy learning limit harm during learning [1, 4, 62]. Persistent self-improvement adds the possibility that an exploratory mistake becomes reusable material or changes a later constraint. Experiments should jointly report useful discoveries, unsafe exposure, containment failures, and recurrence after repair. A recovery method can restore an old score and still fail. It may remove useful capability, miss a derived artifact, or leave physical harm untreated.

8 Conclusion

As AI systems become increasingly capable of persistent adaptation and recursive self-improvement, safety can no longer be treated solely as a property of a model or system at a single point in time. This work introduced Evolutionary Safety to study how safety changes throughout the processes by which AI systems improve, persist, and produce their successors. We showed why safety becomes evolutionary when changes are retained, selected, inherited, and propagated; characterized its manifestations from intent drift and error accumulation to evaluator drift and risk propagation; and located their mechanisms across persistent agent state, model state, evaluation and environmental feedback, computational substrate, and meta-level update mechanisms with co-evolution. We further developed a framework for discovering and evaluating these risks across states, updates, trajectories, and lineages, and discussed governance through modification boundaries, independent evaluation and authorization, provenance, and recovery. These perspectives also expose important open problems in long-horizon safety preservation, emerging-risk discovery,

cross-artifact and population propagation, effective recovery, and scalable oversight. Ultimately, as AI increasingly participates in shaping its own development, ensuring safety requires not only evaluating what a system is, but understanding and governing how it continues to change.

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained Policy Optimization. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. 22–31.
- [2] Sina Alemohammad, Josue Casco-Rodriguez, Lorenzo Luzi, Ahmed Imtiaz Humayun, Hossein Babaei, Daniel LeJeune, Ali Siahkoobi, and Richard G. Baraniuk. 2024. Self-Consuming Generative Models Go MAD. In *International Conference on Learning Representations (ICLR)*.
- [3] Lama Alssum, Hani Itani, Hasan Abed Al Kader Hammoud, Philip Torr, Adel Bibi, and Bernard Ghanem. 2025. Unforgotten Safety: Preserving Safety Alignment of Large Language Models with Continual Learning. arXiv:2512.10150 [cs.LG]
- [4] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.
- [5] Hengyu An, Minxi Li, Naen Xu, et al. 2026. PerMemSafe: Benchmarking Implicit Personalized Safety of Long-Horizon Self-Evolving Agents. In *Findings of the Association for Computational Linguistics: ACL*.
- [6] Marcin Andrychowicz, Misha Denil, Sergio Gómez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. 2016. Learning to Learn by Gradient Descent by Gradient Descent. In *Advances in Neural Information Processing Systems*, Vol. 29. 3981–3989.
- [7] Ariel Flint Ashery, Luca Maria Aiello, and Andrea Baronchelli. 2024. Emergent Social Conventions and Collective Bias in LLM Populations. arXiv:2410.08948 [cs.SI]
- [8] Joe Benton, Misha Wagner, Eric Christiansen, Cem Anil, Ethan Perez, Jai Srivastav, Esin Durmus, Deep Ganguli, Shauna Kravec, Buck Shlegeris, Jared Kaplan, Holden Karnofsky, Evan Hubinger, Roger Grosse, Samuel R. Bowman, and David Duvenaud. 2024. Sabotage Evaluations for Frontier Models. arXiv:2410.21514.
- [9] Jan Betley, Daniel Tan, Niels Warncke, Anna Sztyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. 2025. Emergent Misalignment: Narrow Finetuning Can Produce Broadly Misaligned LLMs. In *International Conference on Machine Learning (ICML)*.
- [10] Harshit Bisht, Vinay Kumar, Kevin Maik Jablonka, Mausam, and N. M. Anoop Krishnan. 2026. Agentic AI Scientists Are Not Built For Autonomous Scientific Discovery. arXiv:2605.08956 [cs.AI]
- [11] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. 2022. Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning. *Annual Review of Control, Robotics, and Autonomous Systems* 5 (2022). doi:10.1146/annurev-control-042920-020211
- [12] Florian Carichon, Aditi Khandelwal, Marylou Fauchard, and Golnoosh Farnadi. 2025. The Coming Crisis of Multi-Agent Misalignment: AI Alignment Must Be a Dynamic and Social Process. arXiv:2506.01080 [cs.MA]
- [13] Tommaso Cerruti, Mika Okamoto, and Ansel Kaplan Erol. 2026. Agent Memory Is a Surface for Endogenous Authorization Laundering. arXiv:2609.01836 [cs.AI]
- [14] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. 2025. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *International Conference on Learning Representations (ICLR)*.
- [15] Guoxin Chen, Zhong Zhang, Xin Cong, Fangda Guo, Yesai Wu, Yankai Lin, Wenzheng Feng, Yasheng Wang, et al. 2025. Learning Evolving Tools for Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [16] Jialuo Chen, Lingqi Jiang, Xinhao Deng, Xiaohu Du, et al. 2026. When Experience Becomes Instruction: Trajectory Poisoning in Self-Evolving Agent Skill Systems. arXiv:2608.05563.
- [17] Mingguang Chen, Licheng Wang, and Bo Qu. 2026. Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops. arXiv:2607.07663.
- [18] Qirui Chen, Jingxian Shuai, Shuangwu Chen, Shenghao Ye, Zijian Wen, Xufei Su, Jie Jin, Jiangming Li, Jun Chen, Xiaobin Tan, and Jian Yang. 2026. HardSecBench: Benchmarking the Security Awareness of LLMs for Hardware Code Generation. In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*.
- [19] Yueh-Han Chen, Jiaxin Wen, and Jan Hendrik Kirchner. 2026. Automated Researchers Can Mitigate Well-characterized Alignment Failures. arXiv:2608.28945.
- [20] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [21] Xueqi Cheng, Wen Chen, Huawei Shen, Shiguang Shan, Xue Chen, and Guojie Li. 2025. Intelligent Algorithm Safety: Concepts, Scientific Problems and Prospects. *Bulletin of the Chinese Academy of Sciences* 40, 3 (2025), 419–428.
- [22] Yu Cheng, Yongkang Hu, Jiuan Zhou, Yushuo Zhang, Yihang Chen, Huichi Zhou, Mingang Chen, Zhizhong Zhang, Kun Shao, Yuan Xie, and Zhaoxia Yin. 2026. TAME: A Trustworthy Test-Time Evolution of Agent Memory with Systematic Benchmarking. arXiv:2602.03224 [cs.AI]
- [23] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.

- [24] Mihai Christodorescu, Earlence Fernandes, Ashish Hooda, Somesh Jha, et al. 2025. Systems Security Foundations for Agentic Computing. arXiv:2512.01295.
- [25] Gyounk Chu, Myeongho Jeon, and Eunho Yang. 2026. J-Zero: Unified Challenger–Solver–Judge Co-Evolution from Zero Data. arXiv:2608.26582.
- [26] Alex Cloud, Minh Le, James Chua, Jan Betley, Anna Szyber-Betley, Jacob Hilton, and Owain Evans. 2025. Subliminal Learning: Language Models Transmit Behavioral Traits via Hidden Signals in Data. arXiv:2507.14805.
- [27] Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. 2026. HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks. arXiv:2604.14709.
- [28] Kalyanmoy Deb. 2000. An Efficient Constraint Handling Method for Genetic Algorithms. *Computer Methods in Applied Mechanics and Engineering* 186, 2–4 (2000), 311–338. doi:10.1016/S0045-7825(99)00389-8
- [29] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [30] Zehang Deng, Yongjian Guo, Changzhou Han, Wanlun Ma, Junwu Xiong, et al. 2025. AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways. *Comput. Surveys* 57, 7 (2025).
- [31] Carson Denison, Monte MacDiarmid, Ethan Perez, et al. 2024. Sycophancy to Subterfuge: Investigating Reward-Tampering in Language Models. arXiv:2406.10162.
- [32] Aladin Djuhera, Swanand Ravindra Kadhe, Farhan Ahmed, Syed Zawad, and Holger Boche. 2025. SafeMERGE: Preserving Safety Alignment in Fine-Tuned Large Language Models via Selective Layer-Wise Model Merging. arXiv:2503.17239.
- [33] A. E. Eiben and J. E. Smith. 2015. *Introduction to Evolutionary Computing* (2 ed.). Springer. doi:10.1007/978-3-662-44874-8
- [34] Tom Everitt, Daniel Filan, Mayank Daswani, and Marcus Hutter. 2016. Self-Modification of Policy and Utility Function in Rational Agents. In *Artificial General Intelligence (Lecture Notes in Computer Science, Vol. 9782)*. Springer, 1–11. doi:10.1007/978-3-319-41649-6_1
- [35] Fanghao Fan, Yingjie Xia, and Li Kuang. 2025. SecV: LLM-based Secure Verilog Generation with Clue-Guided Exploration on Hardware-CWE Knowledge Graph. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*.
- [36] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. arXiv:2508.07407.
- [37] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, et al. 2026. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *Transactions on Machine Learning Research* (2026).
- [38] Jifeng Gao, Kang Xia, Yi Zhang, et al. 2026. MemPoison: Uncovering Persistent Memory Threats and Structural Blind Spots in LLM Agents. arXiv:2607.14651.
- [39] Leo Gao, John Schulman, and Jacob Hilton. 2023. Scaling Laws for Reward Model Overoptimization. In *International Conference on Machine Learning (ICML)*.
- [40] Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Dhruv Pai, et al. 2024. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv:2404.01413.
- [41] Chang Gong, Di Yao, Jin Wang, Wenbin Li, Lanting Fang, Yongtao Xie, Kaiyu Feng, Peng Han, and Jingping Bi. 2026. PORCA: Root Cause Analysis with Partially Observed Data. In *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montreal, QC, Canada, May 4-8, 2026*. IEEE, 1099–1112.
- [42] Chang Gong, Chuzhe Zhang, Di Yao, Jingping Bi, Wenbin Li, and YongJun Xu. 2024. Causal Discovery from Temporal Data: An Overview and New Perspectives. *Comput. Surveys* 57, 4 (2024), 1–38. doi:10.1145/3705297
- [43] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. 2025. Accelerating Scientific Discovery with Co-Scientist. arXiv:2502.18864.
- [44] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, Akbir Khan, Julian Michael, Sören Mindermann, Ethan Perez, Linda Petrini, Jonathan Uesato, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, and Evan Hubinger. 2024. Alignment Faking in Large Language Models. arXiv:2412.14093.
- [45] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. 2024. AI Control: Improving Safety Despite Intentional Subversion. In *International Conference on Machine Learning (ICML)*.
- [46] Hasan Abed Al Kader Hammoud, Umberto Michieli, Fabio Pizzati, and Philip Torr. 2024. Model Merging and Safety Alignment: One Bad Model Spoils the Bunch. In *Findings of the Association for Computational Linguistics: EMNLP*.
- [47] Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. 2025. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. *Comput. Surveys* 58, 6 (2025).
- [48] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024). doi:10.1109/TCAD.2024.3383347
- [49] Chia-Yi Hsu, Chun-Che Li, et al. 2024. Safe LoRA: the Silver Lining of Reducing Safety Risks when Fine-tuning Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [50] Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated Design of Agentic Systems. In *International Conference on Learning Representations (ICLR)*.

- [51] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. 2026. R-Zero: Self-Evolving Reasoning LLM from Zero Data. In *International Conference on Learning Representations (ICLR)*.
- [52] Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. 2025. Booster: Tackling Harmful Fine-tuning for Large Language Models via Attenuating Harmful Perturbation. In *International Conference on Learning Representations (ICLR)*.
- [53] Tiansheng Huang, Yiming Yona, et al. 2025. Antidote: Post-fine-tuning Safety Alignment for Large Language Models against Harmful Fine-tuning Attack. In *International Conference on Machine Learning (ICML)*.
- [54] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel Maxwell Ziegler, Tim Maxwell, Newton Cheng, et al. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv:2401.05566.
- [55] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. 2017. Population Based Training of Neural Networks. arXiv:1711.09846.
- [56] Zachary Kenton, Noah Y. Siegel, János Kramár, Jonah Brown-Cohen, Samuel Albanie, Jannis Bulian, Rishabh Agarwal, David Lindner, Yunhao Tang, Noah D. Goodman, and Rohin Shah. 2024. On Scalable Oversight with Weak LLMs Judging Strong LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [57] Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. 2024. Prover-Verifier Games improve legibility of LLM outputs.
- [58] Jon Kutasov, Chloe Loughridge, Yuqi Sun, Henry Sleight, Buck Shlegeris, Tyler Tracy, and Joe Benton. 2025. Evaluating Control Protocols for Untrusted AI Agents. arXiv:2511.02997.
- [59] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, et al. 2024. RewardBench: Evaluating Reward Models for Language Modeling. arXiv:2403.13787.
- [60] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv:2603.28052.
- [61] Joel Lehman and Kenneth O. Stanley. 2011. Abandoning Objectives: Evolution Through the Search for Novelty Alone. *Evolutionary Computation* 19, 2 (2011), 189–223. doi:10.1162/EVCO_a_00025
- [62] Jan Leike, Miljan Martic, Victoria Kravovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. 2017. AI Safety Gridworlds. arXiv:1711.09883.
- [63] Simon Lermen, Charlie Rogers-Smith, and Jeffrey Ladish. 2024. LoRA Fine-tuning Efficiently Undoes Safety Training in Llama 2-Chat 70B. arXiv:2310.20624.
- [64] Gang Li, Wendi Yu, Yao Yao, Wei Tong, Yingbin Liang, Qihang Lin, and Tianbao Yang. 2024. A Retention-Centric Framework for Continual Learning with Guaranteed Model Developmental Safety. arXiv:2410.03955 [cs.LG]
- [65] Xiaoyang Li, Yiqi Wang, Haohui Lu, Zhi Chen, Mo Li, Pingan Song, Mingkai Zheng, and Taotao Cai. 2026. MemTX: Transactional Belief Commit for Stateful Agent Memory.
- [66] Hongqiang Lin, Chao Liu, Xiaofan Bai, Xuan Jin, Yuhong Li, Nenggan Zheng, and Xipeng Cao. 2026. Beyond Endpoint Performance: Process-Level Evaluation of Self-Evolving Agents. arXiv:2609.24663 [cs.AI]
- [67] Ruixiao Lin, Xinhao Deng, Qingming Li, Jianan Ma, Yunhao Feng, Yuqi Qing, Zhenyuan Li, Yechao Zhang, Shiwen Cui, Changhua Meng, Tianwei Zhang, Xingjun Ma, Qi Li, Ke Xu, and Shouling Ji. 2026. Safety in Self-Evolving LLM Agent Systems: Threats, Amplification, and Case Studies. arXiv:2606.23075.
- [68] Yuchen Ling, Shengcheng Yu, Zhenyu Chen, et al. 2026. Toward Secure LLM Agents: Threat Surfaces, Attacks, Defenses, and Evaluation. arXiv:2606.10749.
- [69] Yantao Liu, Zijun Yao, Rui Min, Yixin Cao, Lei Hou, and Juanzi Li. 2025. RM-Bench: Benchmarking Reward Models of Language Models with Subtlety and Style. In *International Conference on Learning Representations (ICLR)*.
- [70] Yedidel Louck. 2026. Securing LLM-Agent Long-Term Memory Against Poisoning: Non-Malleable, Origin-Bound Authority with Machine-Checked Guarantees. arXiv:2606.24322.
- [71] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292.
- [72] Haoran Lu, Luyang Fang, Ruidong Zhang, Xinliang Li, Jiazhong Cai, Huimin Cheng, Lin Tang, Ziyu Liu, Zeliang Sun, Tao Wang, Yingchuan Zhang, et al. 2025. Alignment and Safety in Large Language Models: Safety Mechanisms, Training Paradigms, and Emerging Challenges. arXiv:2507.19672.
- [73] Aengus Lynch, Benjamin Wright, Caleb Larson, Stuart J. Ritchie, Soren Mindermann, Evan Hubinger, Ethan Perez, and Kevin Troy. 2025. Agentic Misalignment: How LLMs Could Be Insider Threats. arXiv:2510.05179.
- [74] Pingchuan Ma, Zhaoyu Wang, Zimo Ji, Yuguang Zhou, Zhantong Xue, Zongjie Li, Shuai Wang, and Xiaoqin Zhang. 2026. AutoSpec: Safety Rule Evolution for LLM Agents via Inductive Logic Programming. arXiv:2606.24245 [cs.AI]
- [75] Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. 2025. Natural Emergent Misalignment from Reward Hacking in Production RL. arXiv:2511.18397 [cs.LG]
- [76] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Yang, Henry Daum'e III, Dan Clark, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- [77] Lakshmi Likhitha Mankali, Jitendra Bhandari, Manaar Alam, Ramesh Karri, Michail Maniatakis, Ozgur Sinanoglu, and Johann Knechtel. 2024. RTL-Breaker: Assessing the Security of LLMs against Backdoor Attacks on HDL Code Generation. arXiv:2411.17569.
- [78] Qinghua Mao, Wanying Qu, Dadi Guo, Leitao Yuan, Qingyu Liu, Yu Li, Guanxu Chen, Yanwei Fu, Xi Lin, Xia Hu, and Dongrui Liu. 2026. SafeEvolve: Harness-Policy Co-Evolution from Agent Experience for Safety Alignment. arXiv:2609.02786 [cs.AI]
- [79] Xutao Mao, Liangjie Zhao, Xiang Zheng, and Cong Wang. 2026. Practice Makes Unsafe: Skill Misevolution in Self-Improving LLM Agents. arXiv:2608.12851.
- [80] Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Stuart Jennings, Freddy Tuxworth, Tolga H. Dur, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. 2026. Quantifying Frontier LLM Capabilities for Container Sandbox Escape. arXiv:2603.02277.
- [81] Samuel Marks, Johannes Treutlein, Trenton Bricken, Jack Lindsey, Jonathan Marcus, Siddharth Mishra-Sharma, Daniel Ziegler, Emmanuel Ameisen, Joshua Batson, Tim Belonax, et al. 2025. Auditing Language Models for Hidden Objectives. arXiv:2503.10965.
- [82] Antoine Marot, Benjamin Donnot, Camilo Romero, Balthazar Donon, Marvin Lerousseau, Luca Veyrin-Forrer, and Isabelle Guyon. 2020. Learning to Run a Power Network Challenge for Training Topology Controllers. *Electric Power Systems Research* 189 (2020), 106635. doi:10.1016/j.epsr.2020.106635
- [83] Alexander Meinke, Bronson Schoen, J  r  my Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. 2024. Frontier Models are Capable of In-context Scheming. arXiv:2412.04984.
- [84] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Jiang, Elmar Songhori, Seth Wang, Eriko Lee, Mederic Johnson, Omkar Pathak, Azade Bae, et al. 2021. A Graph Placement Methodology for Fast Chip Design. *Nature* 594 (2021), 207–212. doi:10.1038/s41586-021-03544-w
- [85] Loc X. Nguyen, Avi Deb Raha, Huy Q. Le, Eui-Nam Huh, Dusit Niyato, and Choong Seon Hong. 2026. Self-Evolving AI for Humanoids: Mechanisms, Safety, and Evaluation of Post-Deployment Self-Improvement. arXiv:2609.13236 [cs.RO]
- [86] Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv:2506.13131.
- [87] Laurent Orseau and Mark Ring. 2011. Self-Modification and Mortality in Artificial Agents. In *Artificial General Intelligence (Lecture Notes in Computer Science, Vol. 6830)*. Springer, 1–10. doi:10.1007/978-3-642-22887-2_1
- [88] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- [89] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. 2022. The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models. (2022).
- [90] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual Lifelong Learning with Neural Networks: A Review. *Neural Networks* 113 (2019), 54–71. doi:10.1016/j.neunet.2019.01.012
- [91] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.
- [92] Nico Pelleriti, Sree Harsha Nelaturu, Zhanke Zhou, Zongze Li, et al. 2026. What Do Evolutionary Coding Agents Evolve? arXiv:2605.20086.
- [93] Ethan Perez, Sam Ringer, Kamile Lukosiute, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, et al. 2023. Discovering Language Model Behaviors with Model-Written Evaluations. In *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, 13387–13434. doi:10.18653/v1/2023.findings-acl.847
- [94] Anh Pham, Mihir Thalanki, Michael Sun, et al. 2025. Preventing Catastrophic Forgetting: Behavior-Aware Sampling for Safer Language Model Fine-Tuning. arXiv:2510.21885.
- [95] Mitchell A. Potter and Kenneth A. De Jong. 2000. Cooperative Coevolution: An Architecture for Evolving Coadapted Subcomponents. *Evolutionary Computation* 8, 1 (2000), 1–29. doi:10.1162/106365600568086
- [96] Xiangyu Qi, Zhihong Zeng, Yi Chen, et al. 2024. Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To. In *International Conference on Learning Representations (ICLR)*.
- [97] Wanying Qu, Qinghua Mao, Yu Li, Jiyao Liu, Xin Zhang, Dadi Guo, Yanxu Zhu, Qingyu Liu, Leitao Yuan, Xi Lin, Shanfeng Zhu, Yanwei Fu, Jing Shao, Xia Hu, and Dongrui Liu. 2026. SHE: Trajectory-driven Safety Harness Evolution for LLM Agents. arXiv:2608.09885 [cs.AI]
- [98] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, R. B. Xiong, Qingfeng Lan, Wenyi Wang, Nanbo Li, Yibo Yang, Mingchen Zhuge, and J  rgen Schmidhuber. 2026. Self-Improvements in Modern Agentic Systems: A Survey. arXiv:2607.13104.
- [99] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. 2025. A Self-Improving Coding Agent. arXiv:2504.15228 [cs.AI]
- [100] Christopher D. Rosin and Richard K. Belew. 1997. New Methods for Competitive Coevolution. *Evolutionary Computation* 5, 1 (1997), 1–29. doi:10.1162/evco.1997.5.1.1
- [101] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR)*.
- [102] J  rgen Schmidhuber. 2007. G  del machines: Fully self-referential optimal universal self-improvers. (2007), 199–226.
- [103] J  rgen Schmidhuber. 2013. PowerPlay: Training an Increasingly General Problem Solver by Continually Searching for the Simplest Still Unsolvable Problem. *Frontiers in Psychology* 4 (2013), 313. doi:10.3389/fpsyg.2013.00313
- [104] Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, Dongrui Liu, and Jing Shao. 2026. Your Agent May Misevolve: Emergent Risks in Self-evolving LLM Agents. In *International Conference on Learning Representations (ICLR)*.
- [105] Zhichao Shi, Shaoling Jing, Yi Cheng, Hao Zhang, Yuanzhuo Wang, Jie Zhang, Huawei Shen, and Xueqi Cheng. 2025. SafetyQuizzer: Timely and Dynamic Evaluation on the Safety of LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for*

- Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 1733–1747. doi:10.18653/v1/2025.naacl-long.85
- [106] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [107] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. 2024. AI Models Collapse when Trained on Recursively Generated Data. *Nature* 631 (2024), 755–759.
 - [108] Saksham Sahai Srivastava and Haoyu He. 2025. MemoryGraft: Persistent Compromise of LLM Agents via Poisoned Experience Retrieval. arXiv:2512.16962.
 - [109] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2025. JudgeBench: A Benchmark for Evaluating LLM-based Judges. In *International Conference on Learning Representations (ICLR)*.
 - [110] ZenGen Team, Ao Xiang, Bi Jingping, Chen Jiahui, Chen Lehan, Chen Yilin, Cheng Xueqi, Fan Yixing, Gan Kairong, Gao Haowen, et al. 2026. ZenGen: Social Mind for LLMs.
 - [111] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887.
 - [112] Guiyao Tie, Pan Zhou, and Lichao Sun. 2025. A Survey of AI Scientists: Surveying the Automatic Scientists and Research. arXiv:2510.23045.
 - [113] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.
 - [114] Kaixiang Wang et al. 2026. OEP: Poisoning Self-Evolving LLM Agents via Locally Correct but Non-Transferable Experiences. arXiv:2605.18930.
 - [115] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. 2019. POET: Open-Ended Coevolution of Environments and Their Optimized Solutions. In *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, 142–151. doi:10.1145/3321707.3321799
 - [116] Tianlu Wang, Ilia Kulikov, Olga Golovneva, Ping Yu, et al. 2024. Self-Taught Evaluators. arXiv:2408.02666.
 - [117] Xing Wang, Xiaoyi Zhang, and Jie Shao. 2026. Auditing Harness Tampering in Self-Improving Agents. arXiv:2609.00069 [cs.AI]
 - [118] Yixu Wang, Xin Wang, Yang Yao, Xinyuan Li, et al. 2026. AgenticEval: Toward Agentic and Self-Evolving Safety Evaluation of Large Language Models. In *Findings of the Association for Computational Linguistics: ACL*.
 - [119] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent Workflow Memory. arXiv:2409.07429.
 - [120] Tianhao Wu, Weizhe Yuan, Olga Golovneva, et al. 2025. Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
 - [121] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. arXiv:2502.12110.
 - [122] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066.
 - [123] Xianglin Yang et al. 2026. Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections. arXiv:2602.15654.
 - [124] Biao Yi, Jiahao Li, Baolei Zhang, Lihai Nie, Tong Li, Tiansheng Huang, and Zheli Liu. 2025. SafeGrad: Gradient Surgery for Safe LLM Fine-Tuning. arXiv:2508.07172.
 - [125] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, et al. 2025. Gödel Agent: A Self-Referential Agent Framework for Recursively Self-Improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.
 - [126] Peipei Yu, Hongcai Zhang, Yonghua Song, Zhenyi Wang, Huiyu Dong, and Liang Ji. 2025. Safe Reinforcement Learning for Power System Control: A Review. *Renewable and Sustainable Energy Reviews* 223 (2025), 116022. doi:10.1016/j.rser.2025.116022
 - [127] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. 2024. Self-Rewarding Language Models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024 (Proceedings of Machine Learning Research, Vol. 235)*. PMLR / OpenReview.net, 57905–57923.
 - [128] Zhaocong Yuan, Adam W. Hall, Siqi Zhou, Lukas Brunke, Melissa Greeff, Jacopo Panerati, and Angela P. Schoellig. 2022. safe-control-gym: A Unified Benchmark Suite for Safe Learning-based Control and Reinforcement Learning. arXiv:2109.06325 [cs.RO]
 - [129] Zelin Zang, Yuhang Song, Aili Wang, Bingo Wing-Kuen Ling, Qi Sun, Zhen Lei, Fuji Yang, Cheng Zhuo, and Jiebo Luo. 2025. The Dawn of Agentic EDA: A Survey of Autonomous Digital Chip Design. arXiv:2512.23189.
 - [130] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2023. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. *CoRR* abs/2310.02304.
 - [131] Jenny Zhang, Shengran Hu, Cong Lu, Jeff Clune, et al. 2026. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. In *International Conference on Learning Representations (ICLR)*.
 - [132] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. 2025. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations (ICLR)*.
 - [133] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. 2026. HyperAgents. arXiv:2603.19461.
 - [134] Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. 2025. Generative Verifiers: Reward Modeling as Next-Token Prediction. In *International Conference on Learning Representations (ICLR)*.
 - [135] Yisong Zhang, Ran Cheng, Guoxing Yi, and Kay Chen Tan. 2026. A Systematic Survey on Large Language Models for Evolutionary Optimization: From Modeling to Solving. *IEEE Computational Intelligence Magazine* (2026), 1–21. doi:10.1109/MCI.2026.3727078

- [136] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-SafetyBench: Evaluating the Safety of LLM Agents. [arXiv:2412.14470](#).
- [137] Andrew Zhao, Yiran Wu, Yang Yue, et al. 2025. Absolute Zero: Reinforced Self-play Reasoning with Zero Data. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [138] Weixiang Zhao, Yichen Zhang, Yingshuo Wang, Yang Deng, Yanyan Zhao, Xuda Zhi, Yongbo Huang, Hao He, Wanxiang Che, Bing Qin, and Ting Liu. 2026. On Safety Risks in Experience-Driven Self-Evolving Agents. In *Findings of the Association for Computational Linguistics: ACL*.
- [139] Bowen Zheng, Ran Cheng, and Kay Chen Tan. 2025. EvoRL: A GPU-accelerated Framework for Evolutionary Reinforcement Learning. *ACM Transactions on Evolutionary Learning and Optimization* (2025). [doi:10.1145/3750053](#)
- [140] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [141] Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. 2023. SOTOPIA: Interactive Evaluation for Social Intelligence in Language Agents. [arXiv:2310.11667](#) [cs.CL]
- [142] Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. 2025. SEAL: Self-Adapting Language Models. In *Advances in Neural Information Processing Systems*, Vol. 38. 82334–82365. [doi:10.52202/085713-2483](#)